Workshop 5 – Structs and malloc

1. Create a function which returns a struct containing the area and perimeter of a circle. The function should take in the radius as it's only parameter.

```c
// Kristina Paudel

#include <stdio.h>

typedef struct{
float area;
float perimeter;
}circle;

circle calc(float r){
circle c;
c.area=3.14*r*r;
c.perimeter=2*3.14*r;
return c;
}

int main(){
float r;
circle result;

printf("Enter radius: ");
scanf("%f",&r);

result=calc(r);

printf("Area = %.2f\n",result.area);
printf("Perimeter = %.2f",result.perimeter);

return 0;
}
```

```
kristina_07@DESKTOP-33BAI0E:~/workshop$ gcc q1.c
kristina_07@DESKTOP-33BAI0E:~/workshop$ ./a.out
Enter radius: 8
Area = 200.96
Perimeter = 50.24kristina_07@DESKTOP-33BAI0E:~/workshop$
```

2. Create a function which returns a struct containing two roots given from the quadratic formula. The function should take 3 integers as parameters named a, b and c.

```c
// Kristina Paudel

#include<stdio.h>
#include<math.h>

typedef struct{float root1;
float root2;
}quadratic;

quadratic calc(int a,int b,int c){
quadratic q;float d=b*b-4*a*c;
if(a==0){q.root1=q.root2=0;return q;}
if(d>=0){
q.root1=(-b+sqrt(d))/(2*a);
q.root2=(-b-sqrt(d))/(2*a);
}else{
q.root1=q.root2=0;
}
return q;
}

int main(){
int a,b,c;quadratic result;
printf("Enter a b c: ");
scanf("%d%d%d",&a,&b,&c);
result=calc(a,b,c);
printf("Root1=%f\nRoot2=%f",result.root1,result.root2);
return 0;
}
```

```
kristina_07@DESKTOP-33BAI0E:~/week5/workshop$ gcc q2.c -o q2 -lm
kristina_07@DESKTOP-33BAI0E:~/week5/workshop$ ./q2
Enter a b c: 4
5
6
Root1=0.000000
Root2=0.000000kristina_07@DESKTOP-33BAI0E:~/week5/workshop$
```

3. Recreate question 2 but returning a pointer (float). Use the example given in the lecture on how to return a pointer to an array of a specific variable type.

```c
//Kristina Paudel

#include<stdio.h>
#include<math.h>

float* calc(int a,int b,int c){
static float roots[2];
float d=b*b-4*a*c;

if(a==0){roots[0]=roots[1]=0;return roots;}

if(d>=0){
roots[0]=(-b+sqrt(d))/(2*a);
roots[1]=(-b-sqrt(d))/(2*a);
}else{
roots[0]=roots[1]=0;
}

return roots;
}

int main(){
int a,b,c;float *result;

printf("Enter a b c: ");
scanf("%d%d%d",&a,&b,&c);

result=calc(a,b,c);

printf("Root1=%f\nRoot2=%f",result[0],result[1]);

return 0;
}
```

```
kristina_07@DESKTOP-33BAI0E:~/week5/workshop$ ./q3
Enter a b c: 1
6
1
Root1=-0.171573
Root2=-5.828427kristina_07@DESKTOP-33BAI0E:~/week5/workshop$
```

4. Write a structure to store the names, salary, and hours of work per day of 5 employees in a company. Write a program to increase the salary depending on the number of hours of work per day as follows and then print the name of all the employees along with their final salaries. To do this, you should create employees within the main, then check whether their hours fit within the boundaries below, if so, increase the salary.

Hours of work per day	**8**	**10**	**>=12**
Increase in salary	**$50**	**$100**	**$150**

Open q4.c ~/week5/worksh

```c
//Kristina Paudel


#include<stdio.h>
#include<string.h>

struct Employee{
char name[50];
float salary;
int hours;
};

struct Employee* updateSalary(struct Employee emp[]){
static struct Employee updated[5];
int i;

for(i=0;i<5;i++){
updated[i]=emp[i];

if(emp[i].hours>=12){
updated[i].salary+=150;
}
else if(emp[i].hours>=10){
updated[i].salary+=100;
}
else if(emp[i].hours>=8){
updated[i].salary+=50;
}
}

return updated;
}

int main(){
struct Employee emp[5];
struct Employee *result;
int i;

for(i=0;i<5;i++){
printf("Enter name salary hours: ");
scanf("%s%f%d",emp[i].name,&emp[i].salary,&emp[i].hours);
}

result=updateSalary(emp);

for(i=0;i<5;i++){
printf("Name=%s Salary=%.2f\n",result[i].name,result[i].salary);
}

return 0;
}
```

```
kristina_07@DESKTOP-33BAI0E:~/week5/workshop$ gcc q4.c -o q4 -lm
kristina_07@DESKTOP-33BAI0E:~/week5/workshop$ ./q4
Enter name salary hours: a
25000
8
Enter name salary hours: b
20000
7
Enter name salary hours: c
16000
8
Enter name salary hours: d
15500
10
Enter name salary hours: e
14000
6
Name=a Salary=25050.00
Name=b Salary=20000.00
Name=c Salary=16050.00
Name=d Salary=15600.00
Name=e Salary=14000.00
kristina_07@DESKTOP-33BAI0E:~/week5/workshop$
```

5. Complete question 4 but using a separate function and pass in the struct which contains the employee information, this will be a pointer. You only need to pass in one struct, not multiple structs (i.e., only one employee). You will need to think back to the lecture regarding pointers to change the original values from the original address where the employee was created.

Open

q5.c
~/week5/workshop

```c
//Kristina Paudel

#include <stdio.h>

// Define the structure
struct Employee {
    char name[50];
    float salary;
    int hoursPerDay;
};

// Function to update salary using a pointer
// This allows us to modify the "original" employee created in main
void updateSalary(struct Employee *emp) {
    if (emp->hoursPerDay >= 12) {
        emp->salary += 150;
    }
    else if (emp->hoursPerDay == 10) {
        emp->salary += 100;
    }
    else if (emp->hoursPerDay == 8) {
        emp->salary += 50;
    }
}

int main() {
    // 1. Create an array of 5 employees
    struct Employee staff[5] = {
        {"Alice", 2500.0, 8},
        {"Bob", 3000.0, 10},
        {"Charlie", 3500.0, 12},
        {"Diana", 2800.0, 7},
        {"Ethan", 3200.0, 14}
    };

    // 2. Loop through and pass the ADDRESS of each employee to the function
    for (int i = 0; i < 5; i++) {
        // Use the & operator to pass the pointer/address
        updateSalary(&staff[i]);
    }

    // 3. Print results to verify original values were changed
    printf("%-15s | %-15s\n", "Name", "Updated Salary");
    printf("-----------------------------------\n");
    for (int i = 0; i < 5; i++) {
        printf("%-15s | $%.2f\n", staff[i].name, staff[i].salary);
    }

    return 0;
}
```

```
kristina_07@DESKTOP-33BAI0E:~/week5/workshop$ gcc q5.c
kristina_07@DESKTOP-33BAI0E:~/week5/workshop$ ./a.out
Name            | Updated Salary
-----------------------------------
Alice           | $2550.00
Bob             | $3100.00
Charlie         | $3650.00
Diana           | $2800.00
Ethan           | $3350.00
kristina_07@DESKTOP-33BAI0E:~/week5/workshop$
```

6. Let us work on the menu of a library. Create a structure containing book information like accession number, name of author, book title and flag to know whether book is issued or not.
   Create a menu in which the following can be done.
   1 - Display book information
   2 - Add a new book
   3 - Display all the books in the library of a particular author
   4 - Display the number of books of a particular title
   5 - Display the total number of books in the library
   6 - Issue a book
   (If we issue a book, then its number gets decreased by 1 and if we add a book, its number gets increased by 1).

5CS046 | Workshop 5

kristina_07@DESKTOP-33BAI0E

q6.c
~/week5/workshop

```c
//Kristina Paudel

#include <stdio.h>
#include <stdlib.h>
#include <string.h>


struct Book {
    int accession_num;
    char title[100];
    char author[100];
    int is_issued;
};


int main() {
    struct Book library[100];
    int book_count = 0;
    int choice;

    while (1) {
        printf("\n--- Library Menu ---\n");
        printf("1 - Display book information\n");
        printf("2 - Add a new book\n");
        printf("3 - Display books of a particular author\n");
        printf("4 - Display number of books of a particular title\n");
        printf("5 - Display total number of books\n");
        printf("6 - Issue a book\n");
        printf("7 - Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        getchar();


        if (choice == 1) {
            for (int i = 0; i < book_count; i++) {
                printf("\nAccession: %d\nTitle: %s\nAuthor: %s\nStatus: %s\n",
                    library[i].accession_num, library[i].title, library[i].author,
                    library[i].is_issued ? "Issued" : "Available");
            }
        }


        else if (choice == 2) {
            printf("Enter accession number: ");
            scanf("%d", &library[book_count].accession_num);
            getchar();
            printf("Enter title: ");
            fgets(library[book_count].title, 100, stdin);
            library[book_count].title[strcspn(library[book_count].title, "\n")] = 0;
            printf("Enter author: ");
            fgets(library[book_count].author, 100, stdin);
            library[book_count].author[strcspn(library[book_count].author, "\n")] = 0;
            library[book_count].is_issued = 0;
            book_count++;
        }


        else if (choice == 3) {
            char search_author[100];
            printf("Enter author name: ");
            fgets(search_author, 100, stdin);
            search_author[strcspn(search_author, "\n")] = 0;
            for (int i = 0; i < book_count; i++) {
                if (strcmp(library[i].author, search_author) == 0) {
                    printf("Title: %s (Accession: %d)\n", library[i].title, library[i].accession_num);
                }
            }
        }
```

kristina_07@DESKTOP-33BAI0E

q6.c
~/week5/workshop

```c
        else if (choice == 2) {
            printf("Enter accession number: ");
            scanf("%d", &library[book_count].accession_num);
            getchar();
            printf("Enter title: ");
            fgets(library[book_count].title, 100, stdin);
            library[book_count].title[strcspn(library[book_count].title, "\n")] = 0;
            printf("Enter author: ");
            fgets(library[book_count].author, 100, stdin);
            library[book_count].author[strcspn(library[book_count].author, "\n")] = 0;
            library[book_count].is_issued = 0;
            book_count++;
        }


        else if (choice == 3) {
            char search_author[100];
            printf("Enter author name: ");
            fgets(search_author, 100, stdin);
            search_author[strcspn(search_author, "\n")] = 0;
            for (int i = 0; i < book_count; i++) {
                if (strcmp(library[i].author, search_author) == 0) {
                    printf("Title: %s (Accession: %d)\n", library[i].title, library[i].accession_num);
                }
            }
        }


        else if (choice == 4) {
            char search_title[100];
            int count = 0;
            printf("Enter title: ");
            fgets(search_title, 100, stdin);
            search_title[strcspn(search_title, "\n")] = 0;
            for (int i = 0; i < book_count; i++) {
                if (strcmp(library[i].title, search_title) == 0) {
                    count++;
                }
            }
            printf("Total books with title '%s': %d\n", search_title, count);
        }


        else if (choice == 5) {
            printf("Total books in library: %d\n", book_count);
        }


        else if (choice == 6) {
            int acc_num, found = 0;
            printf("Enter accession number to issue: ");
            scanf("%d", &acc_num);
            for (int i = 0; i < book_count; i++) {
                if (library[i].accession_num == acc_num && library[i].is_issued == 0) {
                    library[i].is_issued = 1;
                    printf("Book issued successfully.\n");
                    found = 1;
                    break;
                }
            }
            if (!found) printf("Book not found or already issued.\n");
        }


        else if (choice == 7) {
            break;
        }
    }


    return 0;
```

kristina_07@DESKTOP-33BAI0E

```
kristina_07@DESKTOP-33BAI0E:~/week5/workshop$ gcc q6.c
kristina_07@DESKTOP-33BAI0E:~/week5/workshop$ ./a.out

--- Library Menu ---
1 - Display book information
2 - Add a new book
3 - Display books of a particular author
4 - Display number of books of a particular title
5 - Display total number of books
6 - Issue a book
7 - Exit
Enter choice: 1

--- Library Menu ---
1 - Display book information
2 - Add a new book
3 - Display books of a particular author
4 - Display number of books of a particular title
5 - Display total number of books
6 - Issue a book
7 - Exit
Enter choice: 2
Enter accession number: 34
Enter title: math
Enter author: kristina

--- Library Menu ---
1 - Display book information
2 - Add a new book
3 - Display books of a particular author
4 - Display number of books of a particular title
5 - Display total number of books
6 - Issue a book
7 - Exit
Enter choice: 3
Enter author name: kristina
Title: math (Accession: 34)

--- Library Menu ---
1 - Display book information
2 - Add a new book
3 - Display books of a particular author
4 - Display number of books of a particular title
5 - Display total number of books
6 - Issue a book
7 - Exit
```

7. In the lecture you were shown how to use arrays and malloc (). Write a function fibonacci(int size) that returns a dynamically allocated array containing the first “size” Fibonacci numbers. For example, if you call fibonacci(5), it should return an array containing: 0, 1, 1, 2, 3.

Open

q7.c
~/week5/workshop

```c
//Kristina Paudel

#include <stdio.h>
#include <stdlib.h>

/**
 * Generates the first 'size' numbers of the Fibonacci sequence.
 * Returns a pointer to the dynamically allocated array.
 */
long* fibonacci(int size) {
    // 1. Handle edge case for size 0 or negative
    if (size <= 0) {
        return NULL;
    }

    // 2. Allocate memory for 'size' integers (using long to avoid early overflow)
    long* arr = (long*)malloc(size * sizeof(long));

    // 3. Check if malloc failed (e.g., out of memory)
    if (arr == NULL) {
        return NULL;
    }

    // 4. Fill the array
    for (int i = 0; i < size; i++) {
        if (i == 0) {
            arr[i] = 0;
        } else if (i == 1) {
            arr[i] = 1;
        } else {
            // Each number is the sum of the previous two
            arr[i] = arr[i - 1] + arr[i - 2];
        }
    }

    return arr;
}

int main() {
    int n = 5;
    long* fib_sequence = fibonacci(n);

    if (fib_sequence != NULL) {
        printf("First %d Fibonacci numbers: ", n);
        for (int i = 0; i < n; i++) {
            printf("%ld%s", fib_sequence[i], (i == n - 1) ? "" : ", ");
        }
        printf("\n");

        // 5. IMPORTANT: Always free dynamically allocated memory
        free(fib_sequence);
    }

    return 0;
}
```

```
kristina_07@DESKTOP-33BAI0E:~/week5/workshop$ gcc q7.c
kristina_07@DESKTOP-33BAI0E:~/week5/workshop$ ./a.out
First 5 Fibonacci numbers: 0, 1, 1, 2, 3
kristina_07@DESKTOP-33BAI0E:~/week5/workshop$
```

8. Write a function called primes(int n) that returns a dynamically allocated array containing the first n prime numbers. For example, if you call primes(5), it should return an array containing: 2, 3, 5, 7, 11.

Open

q8.c
~/week5/workshop

```c
//Kristina Paudel

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

/**
 * Helper function to check if a number is prime.
 */
bool is_prime(int num) {
    if (num < 2) return false;
    for (int i = 2; i * i <= num; i++) {
        if (num % i == 0) return false;
    }
    return true;
}

/**
 * Returns a dynamically allocated array containing the first 'n' primes.
 */
int* primes(int n) {
    if (n <= 0) return NULL;

    // Allocate memory for n integers
    int* prime_arr = (int*)malloc(n * sizeof(int));

    if (prime_arr == NULL) {
        return NULL; // Allocation failed
    }

    int count = 0;
    int candidate = 2;

    // Keep checking numbers until we have found 'n' primes
    while (count < n) {
        if (is_prime(candidate)) {
            prime_arr[count] = candidate;
            count++;
        }
        candidate++;
    }

    return prime_arr;
}

int main() {
    int n = 5;
    int* result = primes(n);

    if (result != NULL) {
        printf("The first %d prime numbers are: ", n);
        for (int i = 0; i < n; i++) {
            printf("%d%s", result[i], (i == n - 1) ? "" : ", ");
        }
        printf("\n");

        // Free the memory to prevent leaks
        free(result);
        result = NULL;
    }

    return 0;
```

```
kristina_07@DESKTOP-33BAI0E:~/week5/workshop$ gcc q8.c
kristina_07@DESKTOP-33BAI0E:~/week5/workshop$ ./a.out
The first 5 prime numbers are: 2, 3, 5, 7, 11
kristina_07@DESKTOP-33BAI0E:~/week5/workshop$
```

9. Write a program that asks the user for n (how many numbers they want to store). Use malloc to create an integer array and let the user enter n numbers into it. Then use malloc again to create a second integer array of the same size, and copy the numbers into it in reverse order. Print and then free() both arrays.

```c
//Kristina Paudel

#include <stdio.h>
#include <stdlib.h>


int main() {
    int n;


    printf("How many numbers do you want to store? ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Please enter a valid positive integer.\n");
        return 1;
    }


    int* first_arr = (int*)malloc(n * sizeof(int));
    if (first_arr == NULL) {
        return 1;
    }


    printf("Enter %d numbers:\n", n);
    for (int i = 0; i < n; i++) {
        printf("Number %d: ", i + 1);
        scanf("%d", &first_arr[i]);
    }


    int* second_arr = (int*)malloc(n * sizeof(int));
    if (second_arr == NULL) {
        free(first_arr);
        return 1;
    }


    for (int i = 0; i < n; i++) {
        second_arr[i] = first_arr[n - 1 - i];
    }


    printf("\nReversed numbers:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", second_arr[i]);
    }
    printf("\n");


    free(first_arr);
    free(second_arr);


    first_arr = NULL;
    second_arr = NULL;


    return 0;
}
```

```
kristina_07@DESKTOP-33BAI0E:~/week5/workshop$ gcc q9.c
kristina_07@DESKTOP-33BAI0E:~/week5/workshop$ ./a.out
How many numbers do you want to store? 6
Enter 6 numbers:
Number 1: 12
Number 2: 34
Number 3: 56
Number 4: 78
Number 5: 90
Number 6: 98

Reversed numbers:
98 90 78 56 34 12
kristina_07@DESKTOP-33BAI0E:~/week5/workshop$ 76
```